Deep Learning Interview Questions and Answers

Simple answers you can say in your own words
AI / ML Engineer interviews

How to use this: Read each question, answer out loud first, then compare with the answer here. The 'Say in interview' box is a short version you can speak in 20 to 30 seconds. These are practice questions based on common topics, not leaked questions from any company.

Part 1: Basics

Q1. What is deep learning? How is it different from machine learning?

Answer:

Deep learning is a part of machine learning that uses neural networks with many layers. In classic machine learning, we often create the features by hand. In deep learning, the network learns the features by itself from raw data, like pixels of an image or words of a text. It needs more data and more computing power, but it works very well on images, speech, and text.

Say in interview: Deep learning uses neural networks with many layers that learn the features by themselves from raw data. It needs more data and computing, but it is very strong for images, speech, and text.

Q2. What is a neural network? Explain the neuron and the layers.

Answer:

A neural network is made of layers of small units called neurons. Each neuron takes inputs, multiplies each by a weight, adds them with a bias, and passes the result through an activation function to the next layer.

- **Input layer:** takes the data.
- **Hidden layers:** learn the patterns. More layers means a deeper network.
- **Output layer:** gives the final prediction.
- The weights and biases are what the network learns during training.

Say in interview: A neural network is layers of neurons. Each neuron multiplies inputs by weights, adds a bias, and applies an activation function. Training adjusts the weights.

Q3. What are activation functions? Name the common ones.

Answer:

An activation function decides the output of a neuron. Without it, the whole network would be just one straight-line calculation, however many layers it has. The activation adds non-linearity, so the network can learn complex patterns.

- **ReLU:** gives 0 for negative values and the same value for positive values. Fast and the most common in hidden layers.
- **Sigmoid:** gives a value between 0 and 1. Used in the output for yes or no problems.
- **Tanh:** gives a value between -1 and 1.

- **Softmax:** turns the outputs into probabilities that add up to 1. Used for multi-class output.

Say in interview: Activation functions add non-linearity. I use ReLU in hidden layers, sigmoid for yes or no output, and softmax for many classes.

Q4. What is the forward pass and backpropagation?

Answer:

In the forward pass, the data goes through the layers and we get a prediction. Then we calculate the loss, which is how wrong the prediction is. In backpropagation, we send this error backward through the layers and calculate how much each weight contributed to the error (the gradient). Then gradient descent changes the weights a little to reduce the error. This repeats many times.

Say in interview: Forward pass makes the prediction. Backpropagation sends the error backward to find each weight's share of it, and then the weights are updated.

Q5. What are loss functions? Which one do you use when?

Answer:

A loss function measures how wrong the model is. Training tries to make it small.

- **Mean squared error:** for regression (predicting a number).
- **Binary cross-entropy:** for yes or no classification.
- **Categorical cross-entropy:** for classification with many classes.

Say in interview: The loss function is the score of how wrong the model is. I use mean squared error for numbers and cross-entropy for classification.

Part 2: Training

Q6. What are batch, mini-batch, and stochastic gradient descent?

Answer:

They differ in how much data we use for each weight update.

- **Batch:** the whole dataset for each update. Stable but slow and needs a lot of memory.
- **Stochastic (SGD):** one example at a time. Fast updates but very noisy.
- **Mini-batch:** a small group, like 32 or 64 examples. This is the common choice, as it balances speed and stability.

Say in interview: Batch uses all data per update, SGD uses one example, and mini-batch uses a small group. In practice we use mini-batch.

Q7. What are epoch, batch size, and iteration?

Answer:

An epoch is one full pass over the whole training data. Batch size is how many examples we use in one update. An iteration is one update step.

Example: If we have 1000 examples and a batch size of 100, then one epoch has 10 iterations.

Say in interview: One epoch is one full pass over the data. Batch size is examples per update. Iterations are the number of update steps.

Q8. What are optimizers? What is Adam?

Answer:

An optimizer is the method that updates the weights using the gradients. Plain SGD uses the same learning rate for every weight. Better optimizers are smarter:

- **Momentum:** remembers the previous direction, so the movement is smoother and faster.
- **Adam:** combines momentum with a separate adaptive learning rate for each weight. It works well in most cases and is the common default.

Say in interview: An optimizer updates the weights. Adam combines momentum and adaptive learning rates, and it is a good default choice.

Q9. What is the learning rate? What happens if it is too high or too low?

Answer:

The learning rate is the size of each weight update step. If it is too high, the loss jumps around or even goes up, and training does not settle. If it is too low, training is very slow and may get stuck. We often start high and reduce it slowly during training, using a learning rate schedule.

Say in interview: Too high and the training becomes unstable. Too low and it is very slow. I tune it and often reduce it during training.

Q10. What are vanishing and exploding gradients? How do you fix them?

Answer:

In a deep network, the gradients are multiplied layer after layer while going backward. If they keep getting smaller, they become almost zero in the early layers, and those layers stop learning. This is the vanishing gradient. If they keep getting bigger, the weights blow up. This is the exploding gradient.

- **Fixes for vanishing:** use ReLU instead of sigmoid or tanh, good weight initialization, batch normalization, and residual connections.
- **Fix for exploding:** gradient clipping, which limits the size of the gradient.
- For sequences, use LSTM or GRU instead of the basic RNN.

Say in interview: Vanishing gradients mean the early layers stop learning. I fix it with ReLU, batch norm, good initialization, and skip connections. Exploding gradients are fixed with clipping.

Q11. What is batch normalization?

Answer:

Batch normalization normalizes the inputs of a layer for each mini-batch, so they have a stable mean and spread. This makes training faster and more stable, allows a higher learning rate, and also gives a small regularization effect.

Say in interview: Batch normalization keeps the layer inputs in a stable range, so training is faster and more stable.

Q12. What is weight initialization and why does it matter?

Answer:

Before training, the weights must start with some values. If all are zero, all neurons learn the same thing. If the values are too big or too small, the gradients can explode or vanish. So we use smart random starting values, like Xavier (Glorot) for sigmoid or tanh, and He initialization for ReLU.

Say in interview: Weights must start with the right random values. Bad starting values cause vanishing or exploding gradients. I use Xavier or He initialization.

Q13. How do you reduce overfitting in deep learning?

Answer:

Overfitting is when the network does very well on training data but badly on new data.

- **More data** and **data augmentation** (for images: flip, rotate, crop).
- **Dropout:** randomly switch off some neurons during training. It is turned off during testing.
- **Early stopping:** stop when the validation error starts to increase.
- **Weight decay** (L2 regularization), a smaller network, and batch normalization.
- **Transfer learning** when the data is small.

Say in interview: I use more data, augmentation, dropout, early stopping, weight decay, and transfer learning.

Part 3: Architectures

Q14. What is a CNN? How does it work?

Answer:

A CNN (Convolutional Neural Network) is made for images. Instead of looking at all the pixels at once, it uses small filters that slide over the image and detect patterns. The early layers find simple things like edges. The deeper layers combine them into shapes, and then objects.

- **Convolution layer:** filters slide over the image and create feature maps.
- **Pooling layer:** makes the feature maps smaller, keeping the strongest values.
- **Fully connected layer at the end:** gives the final class.

Say in interview: A CNN uses small filters sliding over the image to find edges, then shapes, then objects. Pooling reduces the size, and the final layers give the class.

Q15. Why is a CNN better than a normal neural network for images?

Answer:

A normal network connects every pixel to every neuron, so it needs a huge number of weights, and it ignores the position of the pixels. A CNN shares the same small filter across the whole image, so it needs far fewer weights. It also finds a pattern like an eye or an edge wherever it appears in the image.

Say in interview: A CNN uses far fewer weights because filters are shared, and it can find a pattern anywhere in the image.

Q16. What is an RNN? What is its problem?

Answer:

An RNN (Recurrent Neural Network) is made for sequences, like text, speech, or time series. It reads one item at a time and keeps a hidden state, which is a memory of what it has seen so far. Its problems are that it forgets long-range information (vanishing gradient), and it is slow because it must process the steps one after another, not in parallel.

Say in interview: An RNN reads a sequence step by step with a memory. It forgets long-range information and is slow, because it cannot work in parallel.

Q17. What are LSTM and GRU?

Answer:

They are improved RNNs that solve the forgetting problem. They use gates to control what to keep, what to forget, and what to output. LSTM has a separate memory cell and three gates (input, forget, and output). GRU is a simpler version with fewer gates, so it is faster and has fewer parameters.

Say in interview: LSTM and GRU are RNNs with gates that decide what to remember and what to forget, so they handle long sequences better.

Q18. What is a transformer? What is attention?

Answer:

A transformer is the design behind modern LLMs. Instead of reading step by step, it uses attention, where each word looks at all the other words at the same time and decides which ones matter for its meaning. For example, in 'The dog did not eat the food because it was tired', the word 'it' focuses on 'dog'.

- It works in parallel, so it trains much faster on GPUs than RNNs.
- It handles long-range relations better.
- It needs positional encoding, because attention alone does not know the word order.

Say in interview: A transformer uses attention so every word can look at all the other words at once. It is faster and better than RNNs on long text.

Q19. What is transfer learning?

Answer:

Transfer learning means starting from a model that is already trained on a big dataset, and adapting it to our smaller task. We keep the early layers, which have learned general patterns, replace the last layers, and train them on our data. Sometimes we also fine-tune the whole model with a small learning rate. It saves time and data, and gives better results when our data is small.

Example: Take an image model trained on millions of images, and train it only to tell apart our 5 types of products.

Say in interview: Transfer learning reuses a pretrained model for a new task. It needs less data and time, and often gives better results.

Q20. What are residual connections (ResNet)?

Answer:

A residual (skip) connection adds the input of a block directly to its output, so the information and the gradients can skip layers. This makes it possible to train very deep networks (100 layers or more) without the vanishing gradient problem. Transformers also use skip connections.

Say in interview: Skip connections pass the input directly to a later layer, so very deep networks can be trained without the gradients vanishing.

Q21. What are autoencoders, GANs, and diffusion models?

Answer:

They are models that create or rebuild data.

- **Autoencoder:** an encoder squeezes the input into a small code, and a decoder rebuilds it. Used for compression, removing noise, and finding anomalies.
- **GAN:** two networks. A generator makes fake data and a discriminator tries to catch the fakes. They improve each other.
- **Diffusion model:** starts from random noise and removes the noise step by step until a clear image appears. It is the base of many modern image generators.

Say in interview: An autoencoder compresses and rebuilds data. A GAN has a generator against a discriminator. A diffusion model removes noise step by step to create an image.

Part 4: Practical Questions

Q22. Why do we use GPUs for deep learning?

Answer:

Deep learning is mostly big matrix calculations, which can be done in parallel. A GPU has thousands of small cores that do many such calculations at the same time, so it is much faster than a normal CPU for training and running neural networks.

Say in interview: Deep learning needs a huge number of parallel matrix calculations, and a GPU has thousands of cores that do exactly that.

Q23. What are hyperparameters? How do you tune them?

Answer:

Hyperparameters are the settings we choose before training, not learned by the model, like the learning rate, batch size, number of layers, dropout rate, and number of epochs. We tune them using the validation set, with grid search, random search, or smarter tools like Bayesian optimization. Random search is usually more efficient than grid search.

Say in interview: Hyperparameters are the settings we choose before training, like the learning rate and batch size. I tune them on the validation set using random search or similar tools.

Q24. What is model compression? (quantization, pruning, distillation)

Answer:

Big models are slow and costly, so we make them smaller and faster.

- **Quantization:** store the weights with fewer bits, like 8-bit or 4-bit instead of 32-bit.
- **Pruning:** remove the weights or neurons that are not important.
- **Distillation:** train a small student model to copy the behavior of a big teacher model.

Say in interview: Quantization uses fewer bits, pruning removes unimportant weights, and distillation trains a small model to copy a big one.

Q25. PyTorch vs TensorFlow. Which one do you use?

Answer:

Both are deep learning frameworks. PyTorch is very popular in research and for modern models like LLMs. It is easy to write and debug, and feels like normal Python. TensorFlow, with Keras, is also widely used, especially for production and mobile deployment. For a new project today, many teams choose PyTorch. Say which one you really have used.

Say in interview: Both are good. PyTorch is popular for research and modern models, and it is easy to debug. TensorFlow is strong for production. I choose by the project and the team.

Q26. Your model is not learning (the loss is not going down). What do you check?

Answer:

I check step by step, from the simple things:

- **Data:** are the labels correct? Is the data scaled? Is there a bug in the data loading?
- **Learning rate:** try a smaller or bigger one.
- **Try to overfit a tiny dataset first.** If the model cannot even do that, there is a bug in the code or the model.
- **Gradients:** check for vanishing or exploding values, and use gradient clipping or batch norm if needed.
- **Loss function and output layer:** are they correct for the task?
- Start with a simple model, then make it bigger.

Say in interview: I check the data and labels first, then the learning rate, then try to overfit a tiny dataset. Then I check the gradients and the loss function.

If you do not know an answer, say honestly: 'I have not worked on this directly, but my understanding is...' and then explain what you know. It is better than guessing.